

Deadlock Team Comp Analyzer - technische flow en code-uitleg

Dit document legt het volledige project uit alsof je het tijdens het examen stap voor stap moet verdedigen. Het doel is niet alleen tonen welke commando's je kan uitvoeren, maar vooral waarom de bestanden bestaan, hoe de data door het project stroomt, en hoe het PyTorch-model wiskundig werkt.

De kern van het project is:

```
Deadlock match API
-> ruwe match metadata
-> gefilterde JSONL per patch
-> account IDs
-> Statlocker ppScore cache
-> vlakke trainings-CSV
-> PyTorch neural network
-> opgeslagen .pt checkpoint + .json metadata
-> Streamlit webapp voor voorspellingen
```

De huidige hoofdpatch in de code is `2026-05-22`. Het geselecteerde model voor de app staat in:

```
models/2026-05-22/neural_teamcomp_heroes_ppscore_context.pt
models/2026-05-22/neural_teamcomp_heroes_ppscore_context.json
```

Dit model gebruikt hero picks en Statlocker ppScore-context.

1. Projectdoel

Het project probeert een vraag te beantwoorden:

Gegeven twee Deadlock teams van zes heroes, eventueel met rank/ppScore-informatie, wat is de kans dat team 2 wint?

Dat is een supervised binary classification probleem. Elke historische match levert:

- `x`: inputfeatures, zoals hero IDs en ppScores.
- `y`: targetlabel, namelijk welk team gewonnen heeft.

In de trainingscode is `y = 1` als team 2 wint en `y = 0` als team 1 wint.

De app draait daarna inference: de gebruiker kiest zelf twee teams en ranks, en het model geeft een kans terug.

2. Belangrijkste mappen

```
app/
  streamlit app.py          Webapp waarmee je teams samenstelt en voorspellingen
  ziet.
```

assets/hero_images.json	Manifest met hero-afbeeldingen.
assets/heroes/	Lokale hero-afbeeldingen.
data/	
matches 2026-05-22.jsonl	Gefilterde matches van de patch.
accounts 2026-05-22 20k.txt	Account IDs uit de dataset.
pp_scores.json	Cache van Statlocker ppScores per account.
team comp dataset *.csv	Vlakke trainingsdatasets.
models/	
2026-05-22/	Modellen voor de patch van 22 mei 2026.
2026-05-22/experiments/	Experimentele modellen en metadata.
scripts/	
fetch_matches.py	Haalt Deadlock matches binnen.
extract_accounts.py	Haalt unieke account IDs uit matches.
fetch_pp_scores.py	Haalt Statlocker ppScores op.
build_dataset.py	Bouwt CSV voor training.
train_neural_teamcomp.py	Traint het PyTorch-model.
run_neural_experiments.py	Draait meerdere heroes-only experimenten.
run_ppscore_context_experiments.py	Draait meerdere rank-aware experimenten.
src/	
deadlock_api.py	Deadlock API client.
statlocker_api.py	Statlocker API client.
match_parser.py	Zet API payloads om naar nette matchstructuur.
bulk_extract.py	Haalt matchlijsten uit API-responses.
patch_history.py	Bepaalt patch op basis van starttijd.
env_utils.py	Laadt environment variables uit .env.

De scheiding is bewust:

- `src/` bevat herbruikbare logica.
- `scripts/` bevat CLI-stappen in de pipeline.
- `app/` bevat de user interface.
- `data/` bevat gegenereerde datasets.
- `models/` bevat getrainde resultaten.

3. Centrale paden en helpers

Bestand: `scripts/common.py`

Dit bestand zorgt dat de scripts dezelfde standaardpaden gebruiken. Daardoor moet je niet in elk script opnieuw dezelfde bestandsnamen schrijven.

Belangrijk stuk:

```
REPO_ROOT = Path( file ).resolve().parents[1]
SRC_DIR = REPO_ROOT / "src"
if str(SRC_DIR) not in sys.path:
    sys.path.insert(0, str(SRC_DIR))
```

```
DATA DIR = REPO_ROOT / "data"
CURRENT_PATCH = "2026-05-22"
MATCHES_PATH = DATA DIR / f"matches {CURRENT_PATCH}.jsonl"
ACCOUNTS_PATH = DATA DIR / f"accounts {CURRENT_PATCH}.txt"
PP_SCORES_PATH = DATA DIR / "pp_scores.json"
DATASET_PATH = DATA DIR / f"team_comp_dataset {CURRENT_PATCH}.csv"
FETCH_STATE_PATH = DATA DIR / f"fetch_state_{CURRENT_PATCH}.json"
```

`SRC\_DIR` wordt aan `sys.path` toegevoegd zodat scripts gewoon kunnen importeren:

```
from deadlock_api import DeadlockApiClient
from match_parser import build_match_view
```

De helperfuncties zijn klein maar belangrijk:

```
def read_jsonl(path: Path) -> list[dict[str, Any]]:
    ...

def append_jsonl(path: Path, rows: list[dict[str, Any]]) -> None:
    ...

def load_json(path: Path, default: Any) -> Any:
    ...

def write_json(path: Path, payload: Any) -> None:
    ...

def chunked[T](values: list[T], size: int) -> list[list[T]]:
    return [values[index : index + size] for index in range(0, len(values), size)]
```

JSONL betekent: één JSON-object per regel. Dat is handig voor matchdata omdat je gemakkelijk kan append'en zonder telkens het hele bestand opnieuw te schrijven.

4. Stap 1 - matches ophalen

Bestanden:

- `scripts/fetch\_matches.py`
- `src/deadlock\_api.py`
- `src/bulk\_extract.py`
- `src/match\_parser.py`
- `src/patch\_history.py`

Een typisch commando:

```
cd /root/DL-team-comp-analyzer

.venv/bin/python scripts/fetch_matches.py \
  --output data/matches 2026-05-22.jsonl \
  --state-file data/fetch_state_2026-05-22.json \
```



```
--target-count 20000 \  
--batch-size 100 \  
--game-mode normal \  
--required-patch 2026-05-22 \  
--order-direction desc \  
--sleep-seconds 6.2
```

4.1 Wat doet `fetch\_matches.py`?

Het script vraagt batches matches op bij de Deadlock API. Per batch stuurt het queryparameters mee:

```
query = {  
    "limit": args.batch size,  
    "order by": "match id",  
    "order direction": args.order direction,  
    "min match id": current min match id,  
    "max match id": current max match id,  
    "include info": True,  
    "include player info": True,  
    "include player items": False,  
    "include player stats": False,  
    "include player death details": False,  
    "include objectives": False,  
    "include mid boss": False,  
    "game mode": args.game mode,  
}
```

Belangrijk: `--game-mode normal` zorgt dat Street Brawl niet in de dataset terechtkomt. Dat was een belangrijk probleem eerder in het project. De dataset moet gewone Deadlock-matches bevatten, geen eventmode.

Het script houdt ook bestaande match IDs bij:

```
existing match ids = {str(match.get("match id")) for match in read_jsonl(args.output)}  
}
```

Daardoor worden duplicaten niet opnieuw toegevoegd.

4.2 Resume/state mechanisme

Na elke batch schrijft het script een state-file:

```
write_json(  
    args.state file,  
    {  
        "next min match id": current min match id,  
        "next max match id": current max match id,  
        "last batch size": len(raw matches),  
        "total added this run": added,  
        "total_matches": len(existing_match_ids),  
    },  
)
```

```
    },  
)
```

Als een fetch stopt door een timeout, kan je verdergaan met `--resume`. Het script weet dan vanaf welke match ID het moet verder zoeken.

4.3 Patchfilter

`normalize\_matches` filtert elke match:

```
if required patch and match.view.patch != required patch:  
    skipped_other_patch += 1  
    continue
```

De patch wordt bepaald in `src/patch\_history.py`. Daar staat onder andere:

```
PatchRelease(  
    patch_name="2026-05-22",  
    effective_from_utc=datetime(2026, 5, 22, 21, 52, 5, tzinfo=UTC),  
    source="official Gameplay Update - 05-22-2026 release time",  
)
```

De functie:

```
def infer_patch_from_start_time(start_time s: int | None) -> tuple[str, str]:  
    ...
```

zet een matchtijd om naar de juiste patch. Dit is belangrijk omdat balance patches de waarde van hero picks veranderen.

5. Deadlock API client

Bestand: `src/deadlock\_api.py`

De client doet de echte HTTP request:

```
def fetch_bulk_match_metadata(self, **query_params: Any) -> Any:  
    query = encode_query_params(query_params)  
    url = f"{self.game_api_base}/matches/metadata"  
    if query:  
        url = f"{url}?{query}"  
    return self._get_json(url)
```

De `\_get\_json` functie handelt HTTP-errors af:

```
except HTTPError as exc:  
    retry_after_seconds = extract_retry_after_seconds(exc, response_body)  
    if exc.code == 429:  
        raise DeadlockRateLimitError(...)   
    raise DeadlockApiError(...)
```

Dat betekent:

- HTTP 429 wordt gezien als rate limit.
- Timeouts of netwerkproblemen worden nette Python exceptions.
- Het fetch-script kan daarna wachten en opnieuw proberen.

Dit maakt de pipeline robuuster dan een simpele `urlopen` zonder error handling.

6. Match parsing

Bestand: `src/match\_parser.py`

De API-response is niet direct geschikt voor training. `build\_match\_view` zet een ruwe match om naar een vaste vorm:

```
@dataclass
class PlayerView:
    player name: str
    account id: str
    hero id: str
    hero name: str
    pp score: str
    team key: str
    player_slot: int | None
```

```
@dataclass
class MatchView:
    match id: str
    start time s: int | None
    start time utc: str
    patch: str
    patch source: str
    winner: str
    winner team index: int | None
    team 1 average badge: str
    team 2 average badge: str
    team 1 players: list[PlayerView]
    team_2_players: list[PlayerView]
```

Daarna maakt `match\_view\_to\_dict` een JSON-vriendelijke dict:

```
return {
    "match id": match view.match id,
    "start time s": match view.start time s,
    "start time utc": match view.start time utc,
    "patch": match view.patch,
    "winner": match view.winner,
    "winner team index": match view.winner team index,
    "team 1 players": [asdict(player) for player in match view.team 1 players],
    "team 2 players": [asdict(player) for player in match view.team 2 players],
}
```


Dit is wat in `data/matches\_2026-05-22.jsonl` terechtkomt.

7. Account IDs extraheren

Bestand: `scripts/extract\_accounts.py`

Na het ophalen van matches wil je alle unieke spelers kennen. Dat is nodig om ppScores op te vragen bij Statlocker.

Conceptueel doet dit script:

```
lees matches 2026-05-22.jsonl
voor elke match:
    voor elke speler in team 1 players en team 2 players:
        neem account id
schrijf unieke account IDs naar accounts_2026-05-22_20k.txt
```

Waarom uniek? Omdat dezelfde speler in meerdere matches kan voorkomen. Je wil die ppScore maar één keer opvragen.

Voor 20.000 matches zijn er maximaal:

```
20.000 matches * 12 players = 240.000 player slots
```

Maar het aantal unieke accounts is meestal lager, omdat sommige spelers meerdere keren voorkomen.

8. Statlocker ppScores ophalen

Bestanden:

- `scripts/fetch\_pp\_scores.py`
- `src/statlocker\_api.py`
- `.env`
- `.env.example`

Statlocker vereist een API key. Die staat niet in GitHub. De key staat lokaal in `.env`:

```
STATLOCKER_API_KEY=jouw_key
```

`src/statlocker\_api.py` laadt die key:

```
load_repo_env()
self.api_key = api_key or os.getenv("STATLOCKER_API_KEY")
if not self.api_key:
    raise StatlockerApiError(
        "Missing STATLOCKER API KEY. Add it to .env or export it in your shell."
    )
```

De header is:

```

return {
    "Accept": "application/json",
    "User-Agent": "dl-team-comp-analyzer/0.1",
    "X-API-Key": self.api key,
}

```

Dat is dus geen `Bearer` token, maar een `X-API-Key` header.

8.1 Batch profiles

`fetch\_pp\_scores.py` leest account IDs en vergelijkt die met de bestaande cache:

```

pp_scores = {str(key): int(value) for key, value in load_json(args.output, {}).items()}
pending_accounts = [account for account in accounts if account not in pp_scores]

```

Alleen ontbrekende accounts worden opgehaald. Dat is belangrijk voor rate limits.

Per batch:

```

payload = client.fetch_batch_profiles(batch)
fetched = extract_pp_scores(payload)
pp_scores.update(fetched)
write_json(args.output, pp_scores)

```

Na elke batch wordt `data/pp\_scores.json` meteen opgeslagen. Als je het proces stopt, ben je dus niet alles kwijt.

8.2 ppScore extractie

De response wordt verwerkt door:

```

def extract_pp_scores(payload: Any) -> dict[str, int]:
    profiles = payload if isinstance(payload, list) else []
    pp_scores: dict[str, int] = {}
    for profile in profiles:
        account_id = profile.get("accountId") or profile.get("account id")
        pp_score = profile.get("ppScore") or profile.get("pp score")
        ...
        pp_scores[str(account_id)] = int(pp_score)
    return pp_scores

```

`pp\_scores.json` is dus een cache:

```

{
    "123456": 4280,
    "789012": 6125
}

```

9. Dataset bouwen

Bestand: `scripts/build\_dataset.py`

De neural network training verwacht een vlakke CSV. Daarom wordt de geneste JSONL omgezet naar kolommen.

Belangrijke kolommen:

```
BASE COLUMNS = [  
    "match id",  
    "start time s",  
    "start time utc",  
    "patch",  
    "winner",  
    "winner team index",  
    "team 1 average badge",  
    "team 2 average badge",  
    "missing pp scores",  
]
```

Daarna komen hero, account en ppScore kolommen:

```
FIELDNAMES = [  
    *BASE COLUMNS,  
    *[f"team 1 hero {index}" for index in range(1, 7)],  
    *[f"team 2 hero {index}" for index in range(1, 7)],  
    *[f"team 1 account {index}" for index in range(1, 7)],  
    *[f"team 2 account {index}" for index in range(1, 7)],  
    *[f"team 1 pp score {index}" for index in range(1, 7)],  
    *[f"team 2 pp score {index}" for index in range(1, 7)],  
]
```

Een rij in de CSV is dus één match.

9.1 `missing\_pp\_scores`

Bij het bouwen telt het script hoeveel spelers geen ppScore hebben:

```
if account id and account id not in pp scores:  
    missing_pp_scores += 1
```

Die waarde komt in de CSV. Daardoor kan je twee soorten datasets maken:

1. Alle matches, ook met gedeeltelijke ppScore-informatie.
2. Alleen complete matches, met `--require-complete-pp`.

Voor training bleek de grotere gedeeltelijke dataset nuttiger dan alleen de kleine complete subset, zolang het model ook coverage-features krijgt.

10. Trainingsinput

Bestand: `scripts/train\_neural\_teamcomp.py`

De klasse `TeamCompDataset` zet CSV-rijen om naar tensors.

```
team 1 = [hero to index[row[f"team 1 hero {index}"]] for index in range(1, 7)]
team 2 = [hero to index[row[f"team 2 hero {index}"]] for index in range(1, 7)]
extra = extra features(row, use badge=use badge, use pp score=use pp score)
target = 1.0 if row["winner_team_index"] == "1" else 0.0
```

Belangrijk:

- `team\_1` is een lijst van 6 hero-indexen.
- `team\_2` is een lijst van 6 hero-indexen.
- `extra` bevat optioneel badge- en ppScorefeatures.
- `target` is 1 als team 2 won.

De `\_\_getitem\_\_` functie geeft PyTorch tensors terug:

```
return (
    torch.tensor(team 1, dtype=torch.long),
    torch.tensor(team 2, dtype=torch.long),
    torch.tensor(extra, dtype=torch.float32),
    torch.tensor([target], dtype=torch.float32),
)
```

Hero IDs zijn geen continue getallen met betekenis. Hero ID 77 is niet "beter" of "meer" dan hero ID 1. Daarom worden ze eerst omgezet naar embedding-indexen.

11. Hero vocab en embeddings

De functie `build\_hero\_vocab` maakt een mapping:

```
hero ids = sorted({...}, key=int)
return {hero_id: index for index, hero_id in enumerate(hero_ids)}
```

Bijvoorbeeld:

```
"1" -> 0
"2" -> 1
"77" -> 34
```

Daarna gebruikt het model:

```
self.hero_embedding = nn.Embedding(config.num_heroes, config.embedding_dim)
```

Wiskundig is dit een matrix:

$$E \in \mathbb{R}^{(H \times d)}$$

waar:

- `H` = aantal heroes in de dataset.
- `d` = embedding dimension, bijvoorbeeld 24.

Voor een hero-index `i` is de hero-vector:

```
x_hero = E[i]
```

Die vector wordt mee getraind. Het model leert dus zelf een numerieke representatie voor elke hero.

12. Modelarchitectuur

Bestand: `scripts/train\_neural\_teamcomp.py`

De huidige geselecteerde architectuur is `pool`.

De klasse:

```
class TeamCompNet(nn.Module):
    def __init__(self, config: ModelConfig) -> None:
        super().__init__()
        self.hero_embedding = nn.Embedding(config.num_heroes, config.embedding_dim)
        input_dim = self.encoded_dim(config) + config.extra_feature_dim
        self.network = nn.Sequential(
            nn.Linear(input_dim, config.hidden_dim),
            activation_layer(config.activation),
            nn.Dropout(config.dropout),
            nn.Linear(config.hidden_dim, config.hidden_dim // 2),
            activation_layer(config.activation),
            nn.Dropout(config.dropout),
            nn.Linear(config.hidden_dim // 2, 1),
        )
```

De forward pass:

```
team_1_embeddings = self.hero_embedding(team_1)
team_2_embeddings = self.hero_embedding(team_2)
team_1_mean = team_1_embeddings.mean(dim=1)
team_2_mean = team_2_embeddings.mean(dim=1)
features = self.encode_teams(...)
return self.network(torch.cat(features, dim=1))
```

Voor elk team heb je 6 heroes. Na embedding krijg je:

```
team_1_embeddings: batch_size x 6 x embedding_dim
team_2_embeddings: batch_size x 6 x embedding_dim
```

Bij `embedding\_dim = 24` is één team dus zes vectoren van lengte 24.

13. Pool-features

De `pool` architectuur gebruikt gemiddelde en maximum pooling.

Uit `\_encode\_teams`:


```

features = [
    team 1 mean,
    team 2 mean,
    team 2 mean - team 1 mean,
    torch.abs(team 2 mean - team 1 mean),
    team 1 mean * team 2 mean,
]

```

Daarna voor `pool`:

```

team 1 max = team 1 embeddings.max(dim=1).values
team 2 max = team 2 embeddings.max(dim=1).values
features.extend(
    [
        team 1 max,
        team 2 max,
        team 2 max - team 1 max,
        torch.abs(team 2 max - team 1 max),
    ]
)

```

Daarom is de encoded dimension:

```

if config.architecture == "pool":
    return config.embedding_dim * 9

```

Bij `embedding\_dim = 24`:

```

9 * 24 = 216 teamcomp-features

```

Als `--use-pp-score` actief is, komen er nog 6 extra features bij. Dan is:

```

input_dim = 216 + 6 = 222

```

14. ppScore features

De functie `pp\_score\_features` maakt van individuele ppScores zes numerieke features:

```

return [
    (team 2 mean - team 1 mean) / 10000.0,
    (min(team 2 scores) - min(team 1 scores)) / 10000.0,
    (max(team 2 scores) - max(team 1 scores)) / 10000.0,
    (len(team 2 scores) - len(team 1 scores)) / 6.0,
    coverage,
    lobby mean,
]

```

Betekenis:

1. Gemiddeld ppScore-verschil tussen teams.

2. Verschil tussen laagste ppScore-speler van beide teams.
3. Verschil tussen hoogste ppScore-speler van beide teams.
4. Verschil in hoeveel bekende ppScores beide teams hebben.
5. Coverage: hoeveel van de 12 spelers een ppScore hebben.
6. Lobby mean: gemiddelde ppScore van alle bekende spelers.

De eerste drie worden gedeeld door `10000.0`, zodat de waarden klein blijven. Neural networks trainen stabiel als inputfeatures niet extreem groot zijn.

Voorbeeld:

```
team 1 mean = 3200
team 2 mean = 3600

feature_1 = (3600 - 3200) / 10000 = 0.04
```

`lobby\_mean` is belangrijk omdat het absolute rankniveau meegeeft. Zonder lobby mean ziet het model alleen het verschil tussen teams. Als beide teams van 1000 PP naar 6000 PP gaan, blijft het verschil hetzelfde. Met lobby mean weet het model dat de hele lobby hoger is.

15. Wiskundige uitleg van het model

Een match wordt voorgesteld als input:

```
x = [teamcomp_features, ppScore_features]
```

Het netwerk bestaat uit lagen. Een lineaire laag berekent:

```
z = w x + b
```

waar:

- `x` = inputvector.
- `w` = weights.
- `b` = bias.
- `z` = lineaire output voor activatie.

Daarna komt een activatiefunctie. Bij het geselecteerde model wordt `SiLU` gebruikt:

```
SiLU(z) = z * sigmoid(z)
```

met:

```
sigmoid(z) = 1 / (1 + exp(-z))
```

Een eenvoudige laag ziet er dus zo uit:

```
z_1 = w_1 x + b_1
```

```
a_1 = SiLU(z_1)
```

Daarna volgt dropout en nog een lineaire laag:

```
z_2 = w_2 a_1 + b_2  
a_2 = SiLU(z_2)
```

De laatste laag geeft één logit:

```
z_out = w_3 a_2 + b_3
```

Die logit wordt omgezet naar een kans:

```
p = sigmoid(z_out)
```

In dit project betekent:

```
p = P(team 2 wint | gekozen heroes, ppScores)
```

Als $p = 0.60$, dan zegt het model dat team 2 ongeveer 60% kans heeft volgens wat het uit de dataset geleerd heeft.

16. Loss function

Omdat dit binary classification is, gebruikt het model binary cross entropy. In code:

```
criterion = nn.BCEWithLogitsLoss()
```

Wiskundig:

```
L = -[ y log(p) + (1 - y) log(1 - p) ]
```

waar:

- $y = 1$ als team 2 echt won.
- $y = 0$ als team 1 echt won.
- p = voorspelde kans dat team 2 wint.

Als team 2 won en het model voorspelt $p = 0.90$, is de loss laag.

Als team 2 won en het model voorspelt $p = 0.10$, is de loss hoog.

17. Backpropagation en updates

Training betekent: de weights en biases zo aanpassen dat de loss lager wordt.

De basisvorm van gradient descent is:

```
w' = w - η * ∂L/∂w  
b' = b - η * ∂L/∂b
```


waar:

- ``w`` = huidige weights.
- ``b`` = huidige bias.
- ``w'`` = nieuwe weights na update.
- ``b'`` = nieuwe bias na update.
- ``η`` = learning rate.
- ``∂L/∂w`` = gradient van de loss naar de weights.
- ``∂L/∂b`` = gradient van de loss naar de bias.

In code gebeurt dit hier:

```
optimizer.zero_grad()
logits = model(team 1, team 2, extra)
loss = criterion(logits, target)
if args.l1_lambda > 0:
    loss = loss + args.l1_lambda * l1_penalty(model)
loss.backward()
if args.grad_clip > 0:
    nn.utils.clip_grad_norm_(model.parameters(), args.grad_clip)
optimizer.step()
```

Vertaling:

1. ``optimizer.zero_grad()`` verwijdert oude gradients.
2. ``model(...)`` berekent voorspellingen.
3. ``criterion(...)`` berekent loss.
4. ``loss.backward()`` berekent alle gradients.
5. ``clip_grad_norm_`` voorkomt extreem grote updates.
6. ``optimizer.step()`` past ``w`` en ``b`` aan.

Het project gebruikt AdamW:

```
optimizer = torch.optim.AdamW(
    model.parameters(),
    lr=args.learning_rate,
    weight_decay=args.weight_decay,
)
```

AdamW gebruikt adaptieve learning rates per parameter, maar het concept blijft hetzelfde: parameters worden aangepast in de richting die de loss verlaagt.

18. Regularization

Het model kan overfitten: hoge train accuracy maar lage validation/test accuracy. Daarom gebruikt de code meerdere technieken.

18.1 Dropout

In het netwerk:

```
nn.Dropout(config.dropout)
```

Dropout zet tijdens training willekeurig activiteiten op nul. Daardoor kan het model minder gemakkelijk exacte patronen memoriseren.

18.2 Weight decay

AdamW gebruikt:

```
weight_decay=args.weight_decay
```

Dit remt grote weights af. Conceptueel wordt er een straf toegevoegd voor grote w .

18.3 L1 penalty

In de trainingsloop:

```
if args.l1_lambda > 0:
    loss = loss + args.l1_lambda * l1_penalty(model)
```

De functie:

```
def l1_penalty(model: nn.Module) -> torch.Tensor:
    penalty = torch.zeros(), device=next(model.parameters()).device
    for name, parameter in model.named_parameters():
        if parameter.requires_grad and "bias" not in name:
            penalty = penalty + parameter.abs().sum()
    return penalty
```

Wiskundig:

```
L_total = L_BCE +  $\lambda$  * sum(|w|)
```

waar λ de L1-sterkte is. L1 moedigt kleinere en soms sparse weights aan.

18.4 Early stopping

Het model stopt als validation log loss niet meer verbetert:

```
if validation_metrics["log loss"] < best_validation_log_loss - args.min_delta:
    best_validation_log_loss = validation_metrics["log loss"]
    best_epoch = epoch
    epochs_without_improvement = 0
    best_state = {name: value.detach().cpu().clone() for name, value in model.state_dict().items()}
else:
    epochs_without_improvement += 1
    if epochs_without_improvement >= args.patience:
        print(f"Early stopping at epoch {epoch}; best validation epoch was {best_epoch}")
```

```
h}")  
        break
```

Dit voorkomt dat het model blijft trainen nadat het slechter begint te generaliseren.

19. Train/validation/test split

De functie `split\_rows` splitst de dataset:

```
if split == "time":  
    rows.sort(key=lambda row: int or zero(row.get("start time s")))  
else:  
    random.Random(seed).shuffle(rows)
```

Standaard is `split="time"`. Dat is realistischer: je traint op oudere matches en test op latere matches. Het model moet dus generaliseren naar de toekomst, niet naar willekeurige matches uit dezelfde periode.

De testset wordt achteraan genomen:

```
test size = round(len(rows) * test fraction)  
remaining rows = rows[:-test size]  
validation size = round(len(remaining rows) * validation fraction)  
return remaining rows[:-validation size], remaining rows[-validation size:], rows[-test_size:]
```

Bij 20.000 matches en 20% test zijn dat ongeveer 4.000 testmatches.

20. Swap augmentation

In `TeamCompDataset`:

```
self.samples.append((team 1, team 2, extra, target))  
if augment swap:  
    swapped extra = swap extra features(...)  
    self.samples.append((team_2, team_1, swapped_extra, 1.0 - target))
```

Als een match zegt:

```
team 1 vs team 2 -> team 2 wint
```

dan kan je ook leren:

```
team 2 vs team 1 -> team 1 wint
```

Dat verdubbelt de trainingsvoorbeelden en leert het model symmetrie. De ppScore-difference features worden omgedraaid:

```
swapped.extend(
```



```
[
    -pp features[0],
    -pp features[1],
    -pp features[2],
    -pp features[3],
    pp features[4],
    pp features[5],
]
)
```

Coverage en lobby mean blijven hetzelfde, want die horen bij de match als geheel.

21. Evaluatie

De functie `evaluate` berekent accuracy en log loss:

```
probabilities = torch.sigmoid(logits)
predictions = (probabilities >= 0.5).float()
correct += (predictions == target).sum().item()
total_loss += criterion(logits, target).item()
```

Accuracy:

```
accuracy = correcte voorspellingen / totaal aantal voorspellingen
```

Log loss kijkt niet alleen naar juist/fout, maar ook naar zekerheid. Een verkeerde voorspelling met 99% zekerheid wordt zwaar bestraft.

Daarom is log loss vaak belangrijker dan pure accuracy.

22. Experimenten

Bestanden:

- `scripts/run\_neural\_experiments.py`
- `scripts/run\_ppscore\_context\_experiments.py`

Deze scripts draaien meerdere trainingscommando's na elkaar.

Voor heroes-only:

```
EXPERIMENTS = [
    {
        "name": "pool regularized",
        "embedding dim": "16",
        "hidden dim": "96",
        "dropout": "0.40",
        "architecture": "pool",
        "learning rate": "0.0005",
        "weight decay": "0.002",
        "l1 lambda": "0.0000005",
    },
]
```

```
    ...  
]
```

Voor ppScore-context:

```
EXPERIMENTS = [  
    {  
        "name": "context gelu balanced",  
        "activation": "gelu",  
        "embedding dim": "16",  
        "hidden dim": "96",  
        ...  
    },  
    {  
        "name": "context silu balanced",  
        "activation": "silu",  
        ...  
    },  
]
```

Elk experiment schrijft:

- een `.pt` checkpoint;
- een `.json` metadatafile;
- een summary CSV met resultaten.

De gekozen variant werd gekozen op validation log loss, niet zomaar op de hoogste test accuracy. Dat is belangrijk: de testset moet zo weinig mogelijk gebruikt worden om keuzes te maken.

23. Modelbestanden

Een `.pt` checkpoint bevat:

```
torch.save(  
    {  
        "model state dict": model.state dict(),  
        "config": asdict(config),  
        "hero to index": hero to index,  
        "metrics": {...},  
        "history": history,  
        "training args": vars(args),  
        "best epoch": best epoch,  
    },  
    args.model output,  
)
```

De `.json` metadata bevat dezelfde belangrijke informatie in leesbare vorm:

```
{  
    "best epoch": 13,  
    "config": {
```

```

    "activation": "silu",
    "architecture": "pool",
    "embedding dim": 24,
    "hidden dim": 128,
    "use pp score": true
},
"metrics": {
    "test": {
        "accuracy": 0.6082,
        "log loss": 0.6573
    }
}
}

```

De `.pt` is nodig om inference te doen. De `.json` is nodig om het model te begrijpen en te vergelijken.

24. Streamlit app

Bestand: `app/streamlit\_app.py`

De app laadt het model:

```

MODEL_PATH = REPO_ROOT / "models" / "2026-05-22" / "neural_teamcomp_heroes_ppscore_context.pt"
METADATA_PATH = MODEL_PATH.with_suffix(".json")

```

```

@st.cache_resource
def load_model() -> tuple[TeamCompNet, dict[str, int], dict]:
    checkpoint = torch.load(MODEL_PATH, map_location="cpu", weights_only=False)
    config = ModelConfig(**checkpoint["config"])
    model = TeamCompNet(config)
    model.load_state_dict(checkpoint["model_state_dict"])
    model.eval()
    metadata = json.loads(METADATA_PATH.read_text(encoding="utf-8"))
    return model, checkpoint["hero_to_index"], metadata

```

Belangrijk: de app importeert `ModelConfig`, `TeamCompNet` en `pp\_score\_features` uit het trainingsscript:

```

from train_neural_teamcomp import ModelConfig, TeamCompNet, pp_score_features

```

Daardoor gebruikt de app exact dezelfde featureberekening als training. Dat voorkomt train/inference mismatch.

25. Streamlit inference

De kernfunctie:

```

def predict_team_2_probability(

```



```

    model: TeamCompNet,
    hero to index: dict[str, int],
    team 1: list[str],
    team 2: list[str],
    team 1 scores: list[int],
    team 2 scores: list[int],
) -> float:
    team 1 tensor = torch.tensor([[hero to index[hero id] for hero id in team 1]], dtype=torch.long)
    team 2 tensor = torch.tensor([[hero to index[hero id] for hero id in team 2]], dtype=torch.long)
    score row = {
        **{"team 1 pp score {index}": str(score) for index, score in enumerate(team 1 scores, start=1)},
        **{"team 2 pp score {index}": str(score) for index, score in enumerate(team 2 scores, start=1)},
    }
    extra tensor = torch.tensor([pp score features(score row)], dtype=torch.float32)

    with torch.no_grad():
        logits = model(team 1 tensor, team 2 tensor, extra tensor)
        return float(torch.sigmoid(logits).item())

```

Stap voor stap:

1. Gekozen hero IDs worden omgezet naar training-indexen.
2. Team 1 en team 2 worden tensors van vorm `1 x 6`.
3. ppScores worden in dezelfde CSV-achtige vorm gezet.
4. `pp\_score\_features` maakt de 6 extra features.
5. Het model geeft een logit.
6. `sigmoid` maakt er een kans van.

De UI toont daarna:

```

Team 1 win chance = 1 - p
Team 2 win chance = p

```

26. Ranklabels bij ppScore

De app zet ppScore om naar ranktekst:

```

def pp score rank label(pp score: int) -> str:
    if pp score >= 6600:
        return "Eternus 6+"
    if pp score >= 6000:
        return f"Eternus {(pp score - 6000) // 100 + 1}"

    tier index = min(pp score // 600, len(RANK TIERS) - 1)
    sub rank = (pp score % 600) // 100 + 1
    return f"{RANK_TIERS[tier_index]} {sub_rank}"

```

Volgens het nieuwe Statlocker PP-systeem is elke subrank 100 PP. Elke grote

ranktier bevat 6 subranks, dus 600 PP.

Voorbeelden:

```
0 PP    -> Initiate 1
600 PP  -> Seeker 1
1200 PP -> Alchemist 1
3000 PP -> Emissary 1
4200 PP -> Oracle 1
6000 PP -> Eternus 1
6600 PP -> Eternus 6+
```

27. Hero-afbeeldingen

Bestand:

- `scripts/download\_hero\_assets.py`
- `app/assets/hero\_images.json`
- `app/assets/heroes/`

De app gebruikt:

```
HERO_IMAGE_MANIFEST_PATH = REPO_ROOT / "app" / "assets" / "hero_images.json"
```

en:

```
@st.cache data
def load_hero_images() -> dict[str, Path]:
    if not HERO_IMAGE_MANIFEST_PATH.exists():
        return {}
    payload = json.loads(HERO_IMAGE_MANIFEST_PATH.read_text(encoding="utf-8"))
    images: dict[str, Path] = {}
    for hero_id, metadata in payload.items():
        image_path = REPO_ROOT / "app" / "assets" / str(metadata.get("image", ""))
        if image_path.exists():
            images[str(hero_id)] = image_path
    return images
```

Als er geen afbeelding is, toont de app een fallback met hero ID en naam.

28. Waarom 60% accuracy niet slecht is

Een MOBA-match is ruisachtig. Het model kent alleen:

- hero picks;
- ppScore/rankcontext;
- winnaar.

Het model kent niet:

- lane assignments;
- player hero mastery;

- premades/party;
- builds;
- disconnects;
- match duration;
- in-game economy;
- hero nerfs binnen dezelfde patch;
- smurfs;
- teamcommunicatie;
- actuele vorm van spelers.

Daarom is 60% test accuracy op een latere holdout redelijk. Het betekent niet dat het model "maar een beetje beter dan random" is; het betekent dat hero comp + rankcontext voorspellend zijn, maar niet de hele match verklaren.

29. Commands voor de volledige pipeline

Matches ophalen:

```
cd /root/DL-team-comp-analyzer

.venv/bin/python scripts/fetch matches.py \
  --output data/matches 2026-05-22.jsonl \
  --state-file data/fetch state 2026-05-22.json \
  --target-count 20000 \
  --batch-size 100 \
  --game-mode normal \
  --required-patch 2026-05-22 \
  --order-direction desc \
  --sleep-seconds 6.2
```

Accounts extraheren:

```
.venv/bin/python scripts/extract accounts.py \
  --matches data/matches 2026-05-22.jsonl \
  --output data/accounts_2026-05-22_20k.txt
```

ppScores ophalen:

```
.venv/bin/python scripts/fetch pp scores.py \
  --accounts data/accounts 2026-05-22 20k.txt \
  --output data/pp scores.json \
  --log-file data/fetch pp scores 2026-05-22.log \
  --batch-size 100 \
  --sleep-seconds 36
```

Dataset bouwen:

```
.venv/bin/python scripts/build dataset.py \
  --matches data/matches 2026-05-22.jsonl \
  --pp-scores data/pp_scores.json \
```



```
--output data/team_comp_dataset_2026-05-22_20k.csv
```

Complete ppScore dataset bouwen:

```
.venv/bin/python scripts/build_dataset.py \  
--matches data/matches 2026-05-22.jsonl \  
--pp-scores data/pp_scores.json \  
--output data/team_comp_dataset_2026-05-22_20k_ppscore_complete.csv \  
--require-complete-pp
```

Model trainen:

```
.venv/bin/python scripts/train_neural_teamcomp.py \  
--dataset data/team_comp_dataset_2026-05-22_20k.csv \  
--model-output models/2026-05-22/neural_teamcomp_heroes_ppscore_context.pt \  
--epochs 80 \  
--architecture pool \  
--activation silu \  
--embedding-dim 24 \  
--hidden-dim 128 \  
--dropout 0.40 \  
--learning-rate 0.0004 \  
--weight-decay 0.0015 \  
--l1-lambda 0.0000003 \  
--patience 8 \  
--batch-size 256 \  
--use-pp-score
```

App starten:

```
.venv/bin/streamlit run app/streamlit_app.py \  
--server.address 0.0.0.0 \  
--server.port 8501
```

30. Wat je tijdens het examen kan uitleggen

Als gevraagd wordt "hoe werkt dit?", kan je antwoorden:

1. Ik haal alleen normale Deadlock matches op, gefilterd op patch `2026-05-22`.
2. Ik parse de ruwe API-response naar een vaste JSONL-structuur met teams, heroes, accounts en winnaar.
3. Ik extraheer unieke account IDs.
4. Ik haal Statlocker ppScores op en cache die in `pp\_scores.json`.
5. Ik bouw een CSV waarin elke rij één match is.
6. Ik train een PyTorch neural network met hero embeddings.
7. De teams worden samengevat via mean/max pooling.
8. ppScore wordt toegevoegd als relatieve en absolute rankcontext.
9. Het model wordt gekozen op validation log loss.
10. De Streamlit-app laadt hetzelfde model en gebruikt dezelfde featurefunctie voor inference.

31. Belangrijkste ontwerpkeuzes

Patch-specifiek trainen:

Een hero kan na een patch sterker of zwakker zijn.
Daarom is oude data niet altijd representatief.

Normal game mode:

Street Brawl is een andere spelmodus.
Die mengen met normal matches zou labels vervuilen.

Hero embeddings:

Hero IDs zijn categorieën, geen echte getallen.
Embeddings leren per hero een vectorrepresentatie.

Time split:

Train op oudere matches, test op latere matches.
Dat lijkt meer op echte toekomstige voorspellingen.

ppScore context:

Niet alleen teamverschil telt.
Het absolute lobbyniveau kan ook belangrijk zijn.

32. Beperkingen

Het systeem is nuttig als teamcomp-analyzer, maar het is geen perfecte matchvoorspeller.

Belangrijkste beperkingen:

- De dataset bevat alleen wat de API geeft.
- ppScores zijn niet altijd voor alle spelers beschikbaar.
- Het model weet niet of iemand main/offrole speelt.
- Het model kent geen draftvolgorde.
- De app toont correlaties uit historische data, geen gegarandeerde winrate.

Daarom moet je voorspellingen interpreteren als:

"Volgens vergelijkbare historische patronen lijkt team X sterker."

niet als:

"Team X zal zeker winnen."

33. Korte codekaart

```
scripts/fetch_matches.py
    Startpunt om matchdata binnen te halen.

src/deadlock_api.py
    HTTP client voor Deadlock API.

src/match_parser.py
    Maakt nette MatchView en PlayerView objecten.

src/patch_history.py
    Bepaalt de patch via start time s.

scripts/extract_accounts.py
    Verzamelt unieke account IDs.

src/statlocker_api.py
    HTTP client voor Statlocker met X-API-Key.

scripts/fetch_pp_scores.py
    Vult data/pp_scores.json met account -> ppScore.

scripts/build_dataset.py
    Zet JSONL + ppScores om naar trainbare CSV.

scripts/train_neural_teamcomp.py
    Bevat Dataset, modelarchitectuur, loss, training en evaluatie.

scripts/run_neural_experiments.py
    Batch-runner voor heroes-only modellen.

scripts/run_ppscore_context_experiments.py
    Batch-runner voor rank-aware modellen.

app/streamlit_app.py
    User interface en inference.
```

34. Samenvatting van de ML-kern

Voor een match:

```
x = inputfeatures
y = 1 als team 2 wint, anders 0
```

Eerste laag:

```
z_1 = w_1 x + b_1
a_1 = SiLU(z_1)
```

Tweede laag:


```
z_2 = w_2 a_1 + b_2  
a_2 = SiLU(z_2)
```

Output:

```
z_out = w_3 a_2 + b_3  
p = sigmoid(z_out)
```

Loss:

```
L = -[ y log(p) + (1 - y) log(1 - p) ]
```

Parameterupdate:

```
w' = w - η * ∂L/∂w  
b' = b - η * ∂L/∂b
```

Met regularization:

```
L_total = L_BCE + λ * sum(|w|)
```

Dit is het hart van het deep learning-gedeelte: een neural network met embeddinglaag, pooling, extra rankfeatures, nonlinear activations, loss, backpropagation en parameterupdates.